Cellula Technologies

INTRO TO LLMs - 1

By Omar Ahmed



Content

Recall Transformers

GPT 1

GPT 2

GPT 3

LLAMA series

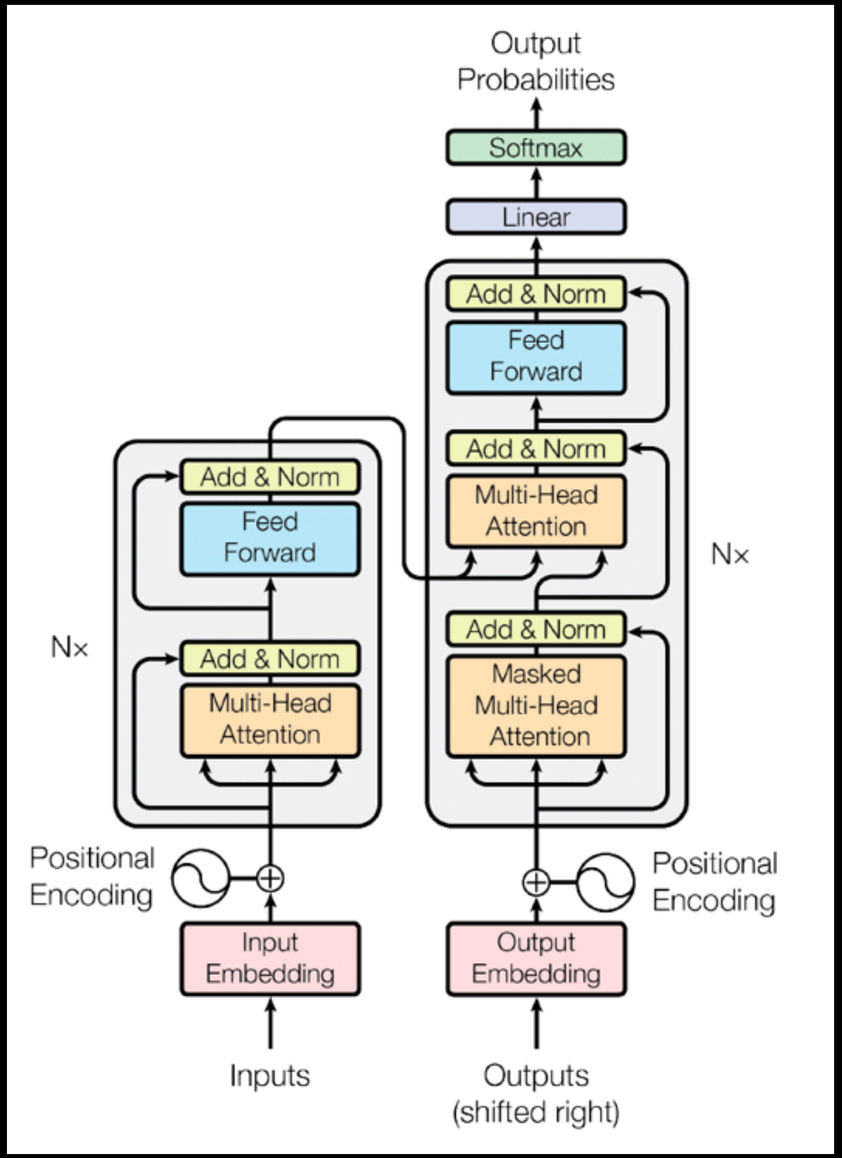
ROPE

Mixture of Experts

Mistral series

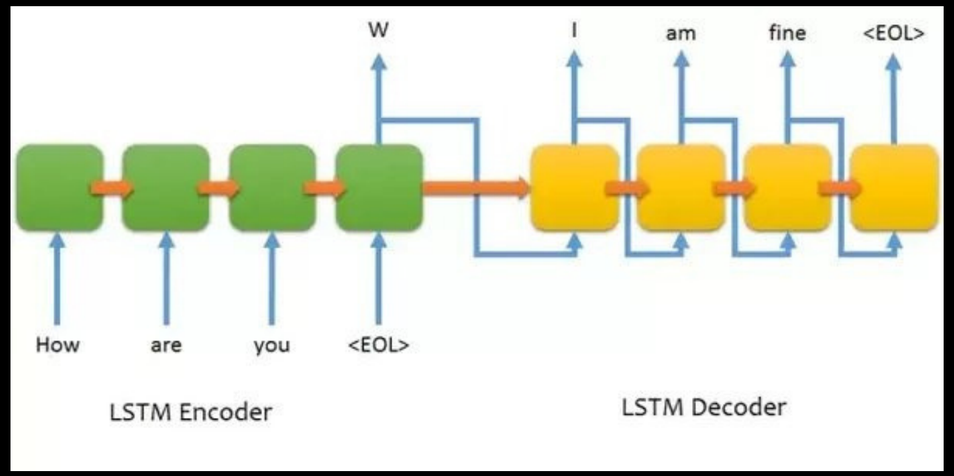
Transformers

- most modern LLMs rely on Transformer Architecture introduced in 2017 paper
- Before Transformers, the main models for text, speech data were RNNs, LSTMs, GRUs, These worked, but they had limitations
- they were slow, hard to parallelize
- Training is hard for very long sequence (vanishing Gradients)
- Transformers solved these problems by using attention



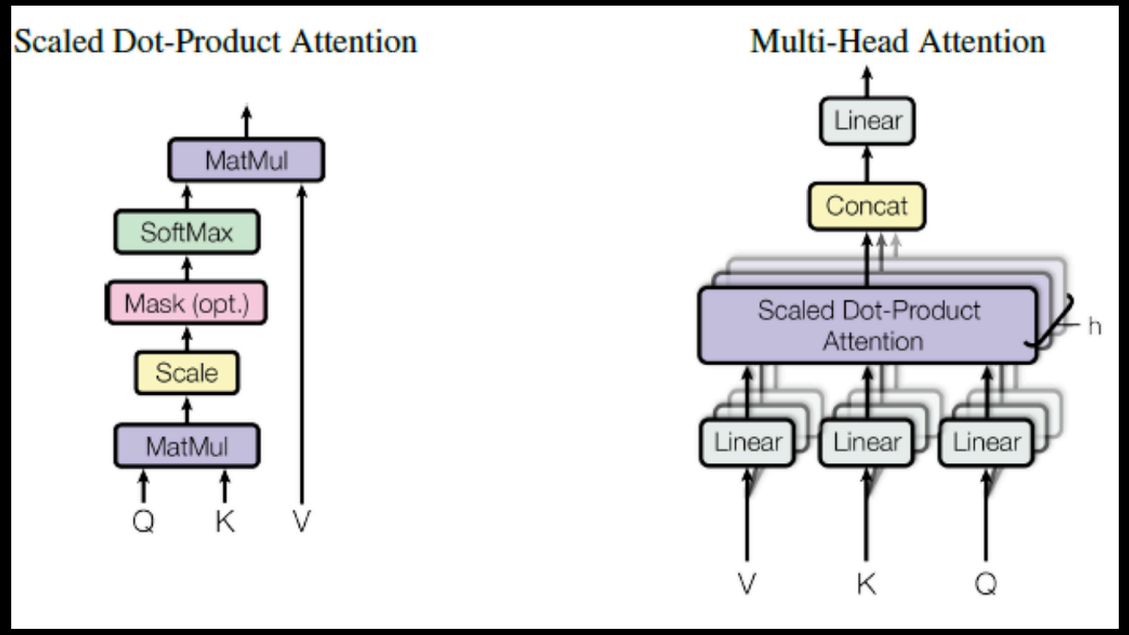
Traditional Seq2Seq model

- Encoder reads the input sequence one token at a time, At each step t, the RNN updates its hidden state
  h_t = tanh(wt.xt + wh.ht-1)
- after the last token, the final hidden state h_t becomes context vector (compressed representation of the entire input)
- There is problem here which is called Bottleneck problem where the entire input sequence is Expected to be captured by a single context vector with fixed dimensionality

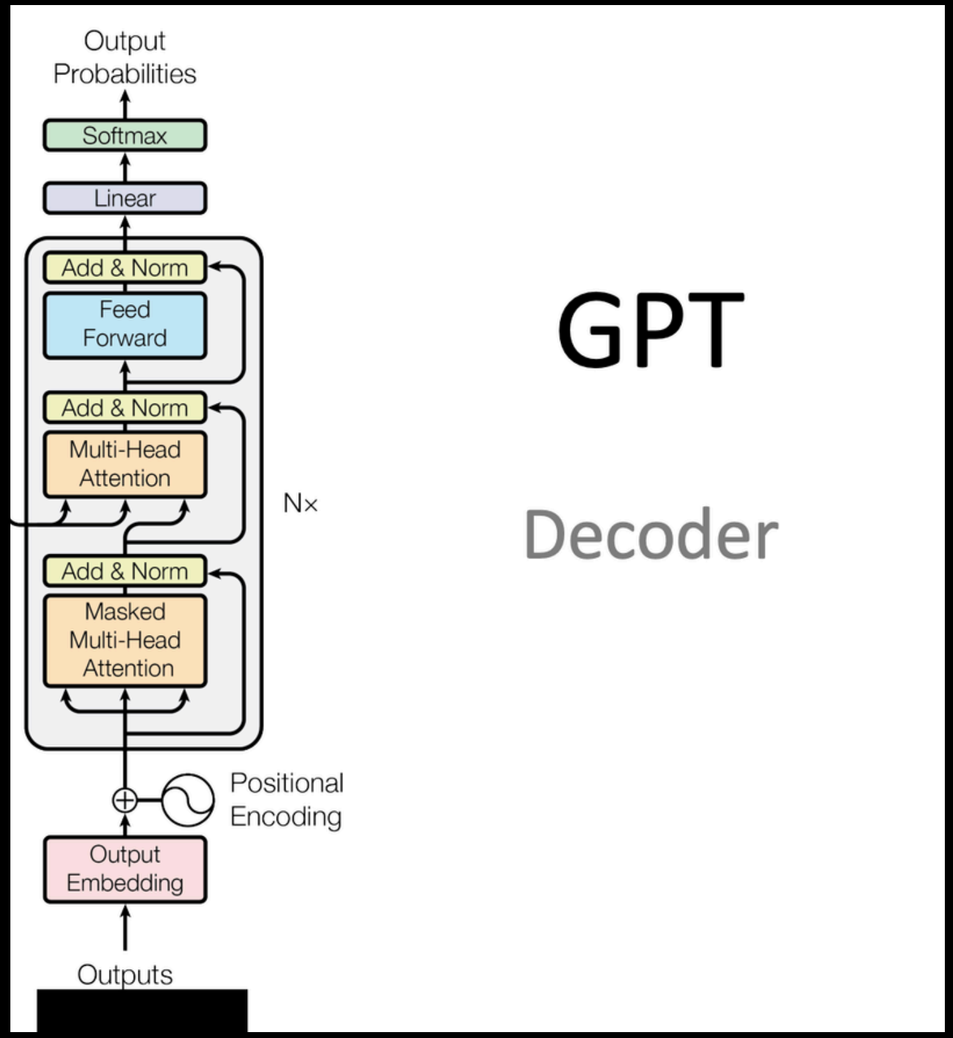


Attention

- Instead of using only one summary vector, attention lets the decoder looks at different parts of the encoder output at each decoding step
- Decide which words in the input are most relevant for predicting the current output word

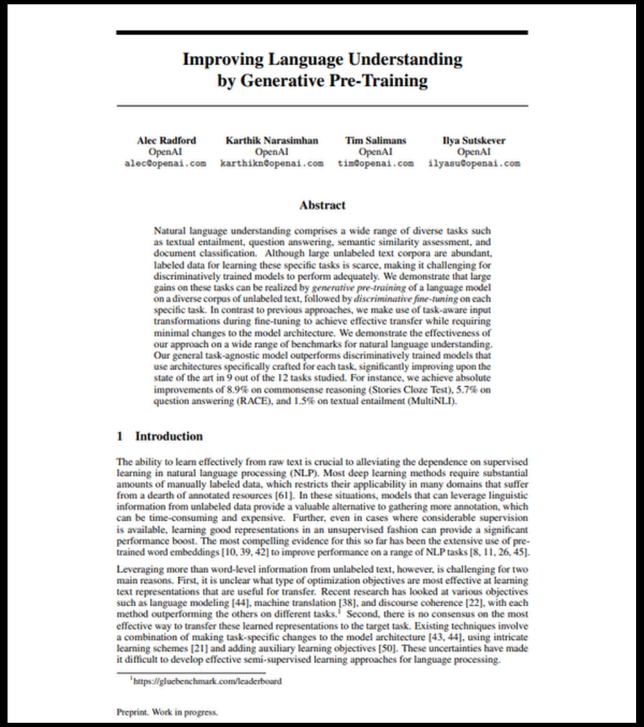


Transformers



GPT Series

- GPT Stands for Generative pre-trained Transformer
- GPT invented by Alec Radford, karthik Narasimhan, Tim Salimans, and Ilya sutskever from openAI
- The first GPT Paper was : *Improving Language understanding by Generative pre-training (Radford et al OpenAI 2018)*



Improving Language Understanding by Generative Pre-Training

Alec Radford OpenAI alec@openai.com
Karthik Narasimhan OpenAI karthikn@openai.com
Tim Salimans OpenAI tim@openai.com
Ilya Sutskever OpenAI ilyasu@openai.com

Abstract

Natural language understanding comprises a wide range of diverse tasks such as textual entailment, question answering, semantic similarity assessment, and document classification. Although large unlabeled text corpora are abundant, labeled data for learning these specific tasks is scarce, making it challenging for discriminatively trained models to perform adequately. We demonstrate that large gains on these tasks can be realized by *generative pre-training* of a language model on a diverse corpus of unlabeled text, followed by *discriminative fine-tuning* on each specific task. In contrast to previous approaches, we make use of task-aware input transformations during fine-tuning to achieve effective transfer while requiring minimal changes to the model architecture. We demonstrate the effectiveness of our approach on a wide range of benchmarks for natural language understanding. Our general task-agnostic model outperforms discriminatively trained models that use architectures specifically crafted for each task, significantly improving upon the state of the art in 9 out of the 12 tasks studied. For instance, we achieve absolute improvements of 8.9% on commonsense reasoning (Stories Cloze Test), 5.7% on question answering (RACE), and 1.5% on textual entailment (MultiNLI).

1 Introduction

The ability to learn effectively from raw text is crucial to alleviating the dependence on supervised learning in natural language processing (NLP). Most deep learning methods require substantial amounts of manually labeled data, which restricts their applicability in many domains that suffer from a dearth of annotated resources [61]. In these situations, models that can leverage linguistic information from unlabeled data provide a valuable alternative to gathering more annotation, which can be time-consuming and expensive. Further, even in cases where considerable supervision is available, learning good representations in an unsupervised fashion can provide a significant performance boost. The most compelling evidence for this so far has been the extensive use of pre-trained word embeddings [10, 39, 42] to improve performance on a range of NLP tasks [8, 11, 26, 45].

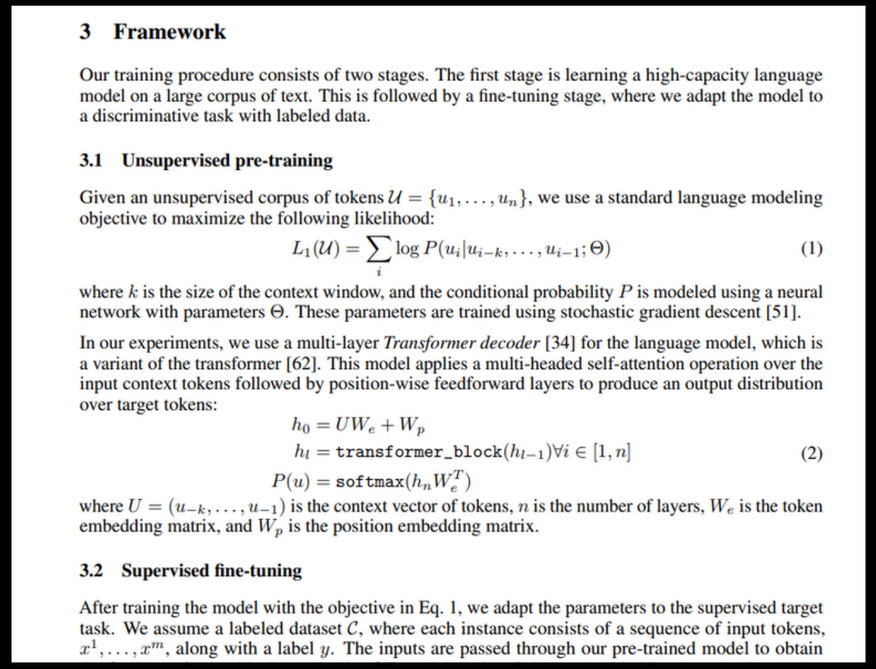
Leveraging more than word-level information from unlabeled text, however, is challenging for two main reasons. First, it is unclear what type of optimization objectives are most effective at learning text representations that are useful for transfer. Recent research has looked at various objectives such as language modeling [44], machine translation [38], and discourse coherence [22], with each method outperforming the others on different tasks.[1] Second, there is no consensus on the most effective way to transfer these learned representations to the target task. Existing techniques involve a combination of making task-specific changes to the model architecture [43, 44], using intricate learning schemes [21] and adding auxiliary learning objectives [50]. These uncertainties have made it difficult to develop effective semi-supervised learning approaches for language processing.

[1] https://gluebenchmark.com/leaderboard

Preprint. Work in progress.

GPT Series

- This was OpenAI's first step into large scale unsupervised language modeling, the seed of what became the modern LLM era
- GPT Never gets a separate file of labels, it is just fed huge chunks of plain text, next word is simply taken from the same text sequence
- No human said after "The sky is very" comes "blue", the dataset itself provides the supervision signal, and that is why it is not called supervised learning, because no external annotation exists, the data teaches itself



3 Framework

Our training procedure consists of two stages. The first stage is learning a high-capacity language model on a large corpus of text. This is followed by a fine-tuning stage, where we adapt the model to a discriminative task with labeled data.

3.1 Unsupervised pre-training

Given an unsupervised corpus of tokens $\mathcal{U} = \{u_1, \ldots, u_n\}$, we use a standard language modeling objective to maximize the following likelihood:

$$L_1(\mathcal{U}) = \sum_i \log P(u_i | u_{i-k}, \ldots, u_{i-1}; \Theta) \quad (1)$$

where k is the size of the context window, and the conditional probability P is modeled using a neural network with parameters Θ. These parameters are trained using stochastic gradient descent [51].

In our experiments, we use a multi-layer *Transformer decoder* [34] for the language model, which is a variant of the transformer [62]. This model applies a multi-headed self-attention operation over the input context tokens followed by position-wise feedforward layers to produce an output distribution over target tokens:

$$\begin{aligned} h_0 &= UW_e + W_p \\ h_l &= \texttt{transformer_block}(h_{l-1}) \forall i \in [1, n] \\ P(u) &= \texttt{softmax}(h_n W_e^T) \end{aligned} \quad (2)$$

where $U = (u_{-k}, \ldots, u_{-1})$ is the context vector of tokens, n is the number of layers, W_e is the token embedding matrix, and W_p is the position embedding matrix.

3.2 Supervised fine-tuning

After training the model with the objective in Eq. 1, we adapt the parameters to the supervised target task. We assume a labeled dataset $\mathcal{C}$, where each instance consists of a sequence of input tokens, $x^1, \ldots, x^m$, along with a label y. The inputs are passed through our pre-trained model to obtain

GPT Series

"The sky is very blue"

We can convert it into a learning problem like this:

Input → Target

"The" → "sky"

"The sky" → "is"

"The sky is" → "very"

"The sky is very" → "blue"

Notice something shocking:

The label already exists in the sentence.

Nobody needed to annotate it.

The sentence supervises itself.

GPT Series

Why this is called unsupervised historically

Historically, machine learning had only two categories:

- **Supervised learning**
- **Unsupervised learning**

Self-supervised learning didn't exist as a formal category.

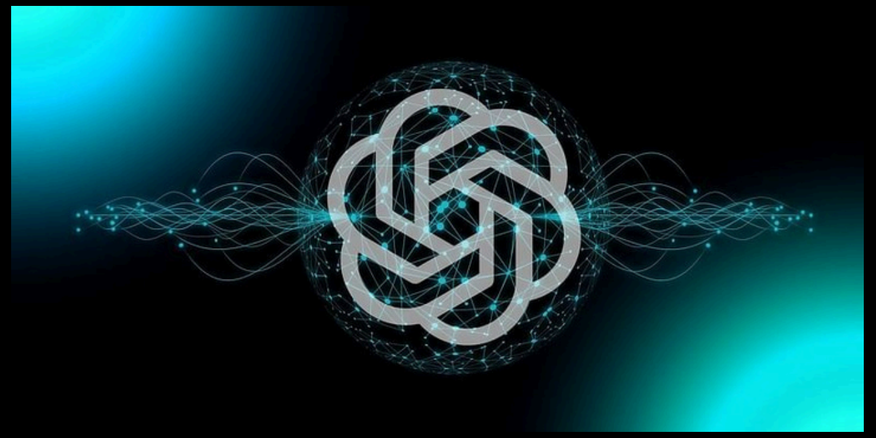
So early papers described it as "unsupervised language modeling"

because no human labels were used.

Modern terminology is more precise:

Self-supervised learning is the correct term.

GPT 1



- 12 layer decoder block
- 12 Attention heads
- embedding dim = 768
- FF inner Dimension 3027
- context window = 512 (maximum number of tokens)
- 117M parameters

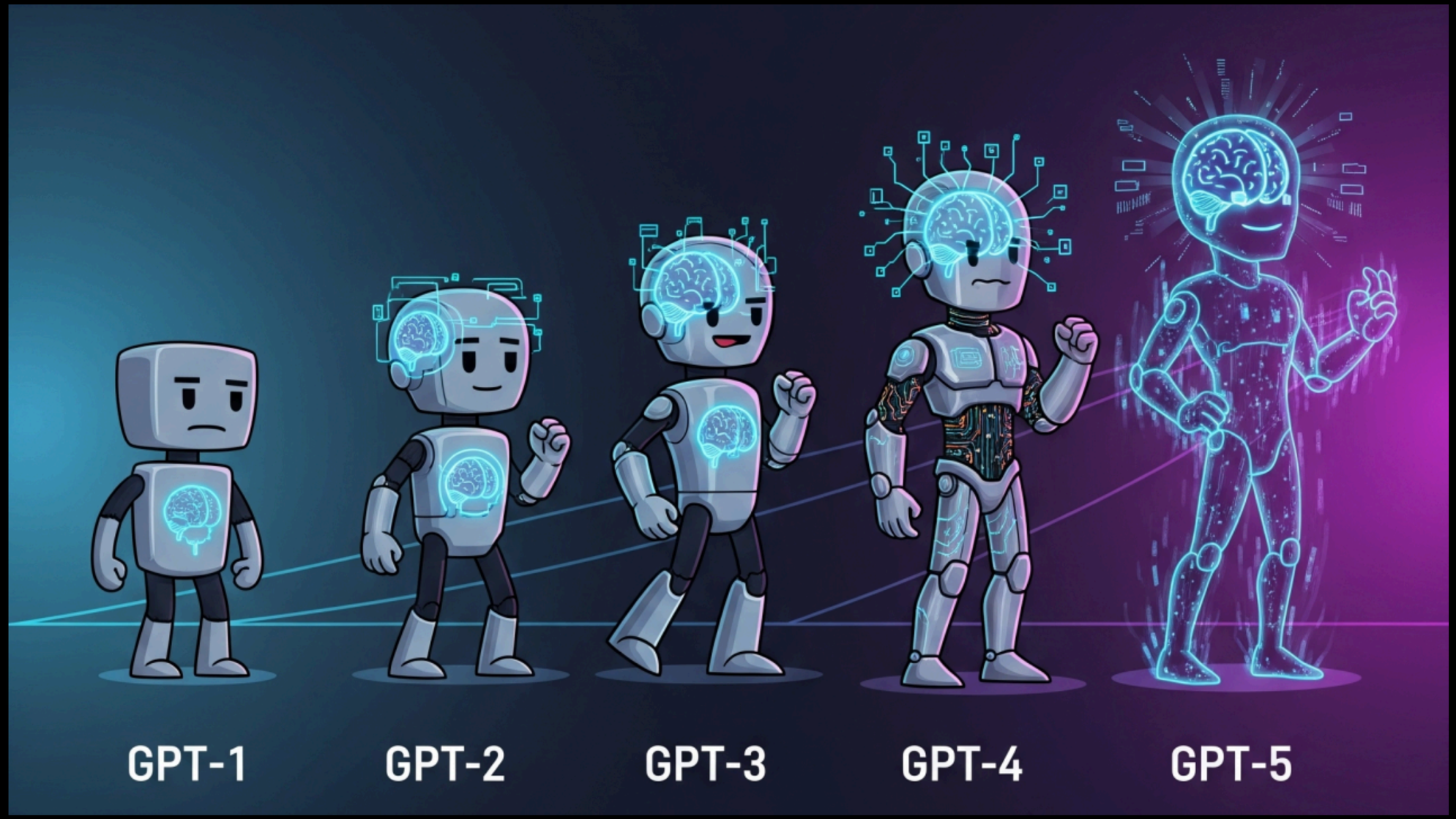
GPT- 1 has 117milions parameters only, that was massive in 2018, but modern models have hunderds of bilions

117M can’t hold much, it memorized some sentence patterns and general grammer, but has no capacity for rich world knowledge or reasoning

GPT 1

- GPT 1 wasn’t meant to be smart, it was a proof of concept that an unsupervised LM trained only to predict the next word, could learn grammer, syntax and a glimmer of semantics without any human labels
- GPT 1 learned only one thing “ predict the next token in a sentence “ , it was never fine tuned to follow instructions, answers Questions, or hold a chat




GPT-1
GPT-2
GPT-3
GPT-4
GPT-5

GPT 2

- **while GPT was essentially proof of concept, GPT 2 was the scaling experiment**
- **OpenAI asked a simple Question “if we make model x10 bigger and give it x100 more text, how far can pure unsupervised learning go**
- **GPT 2 still used same transformer decoder architecture as GPT 1, the idea didn’t change, but scale and training procedure did**

Feature	**GPT 1**	**GPT 2**
Param	117M	1.5B
layers	12	48
hidden	768	1600
attention heads	12	25
context langth	512	1024

GPT 2

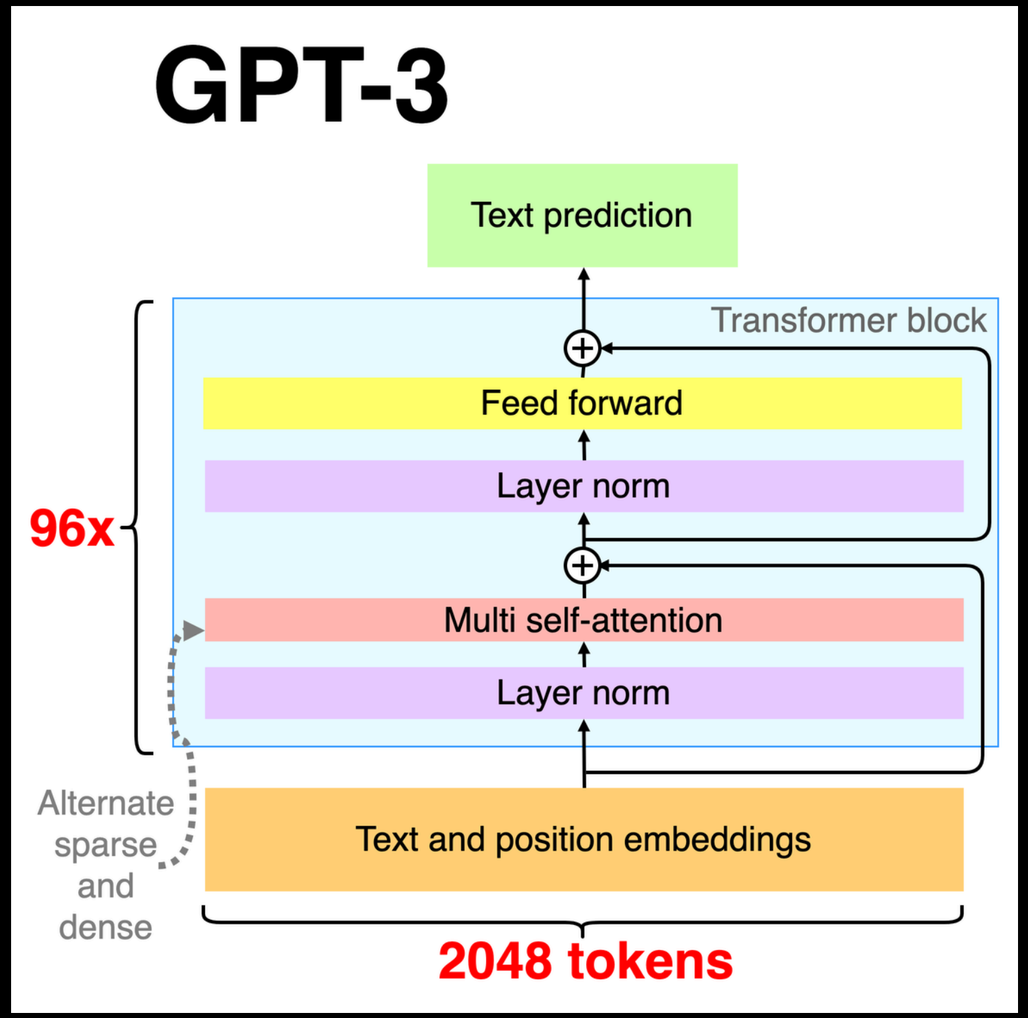
- GPT didn’t understand anything, it learned correlations between words, it could produce beautiful sentences, but if you pushed it logically, it broke instantly
- no internal symbolic reasoning, memory or state representation, it is a probability parrot
- GPT 2 didn’t understand prompts like Explain or summarize, if just continue your text randomly (this because there is no instruction tuning)




AI
OpenAI GPT-3

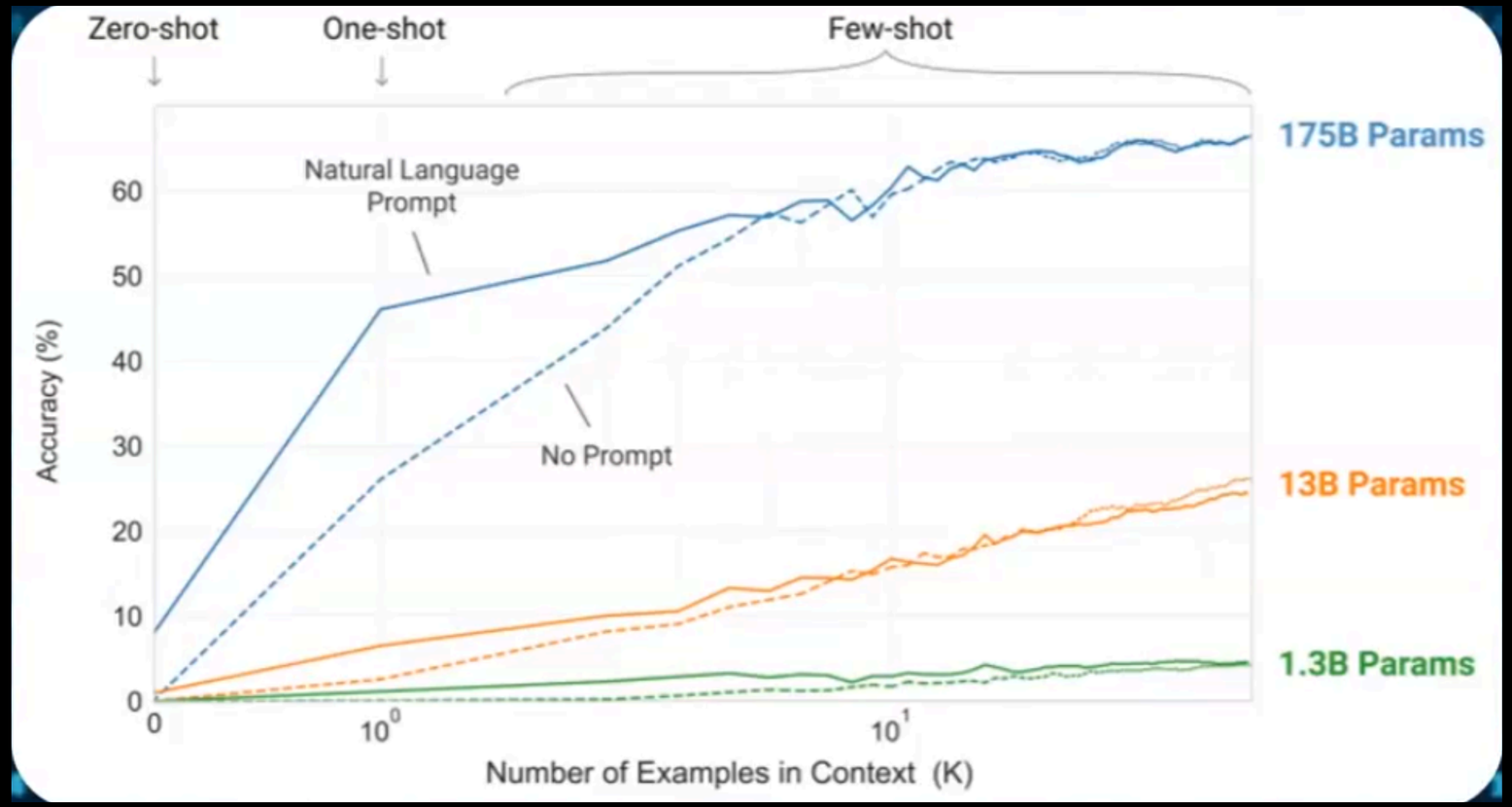
GPT 3

- parametrs : 175B
- layers : 96 layers
- Hidden dimension : 12288
- Attention heads : 96
- context window : 2048 token
- Byte Pair Encoding Tokenization
- trained on 500 billion tokens
- From 1.5B to 175B is x116 times larger size
- This means reasoning ability emerges from sufficient complexity.

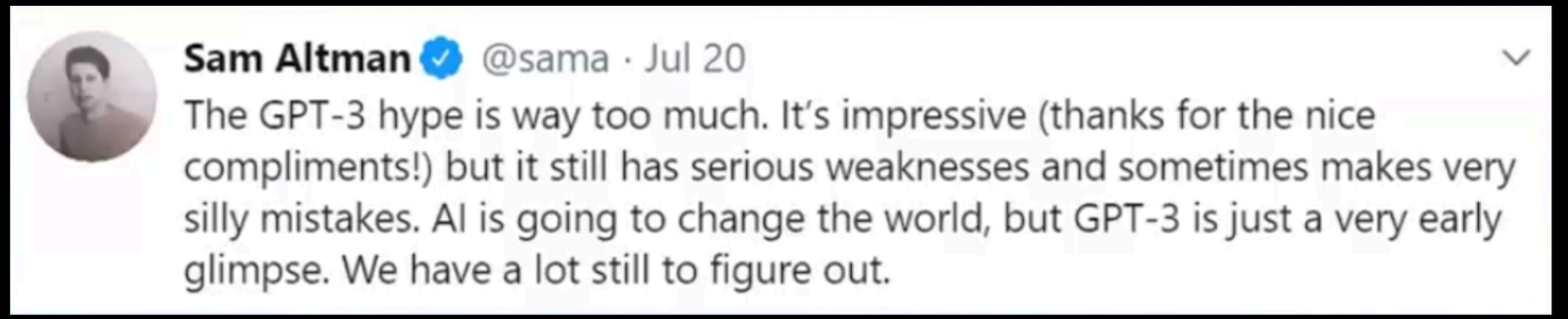


GPT 3

- **GPT-3 can learn new tasks from examples instantly (one shot - few shot)**



GPT 3


Sam Altman @sama · Jul 20
The GPT-3 hype is way too much. It's impressive (thanks for the nice compliments!) but it still has serious weaknesses and sometimes makes very silly mistakes. AI is going to change the world, but GPT-3 is just a very early glimpse. We have a lot still to figure out.

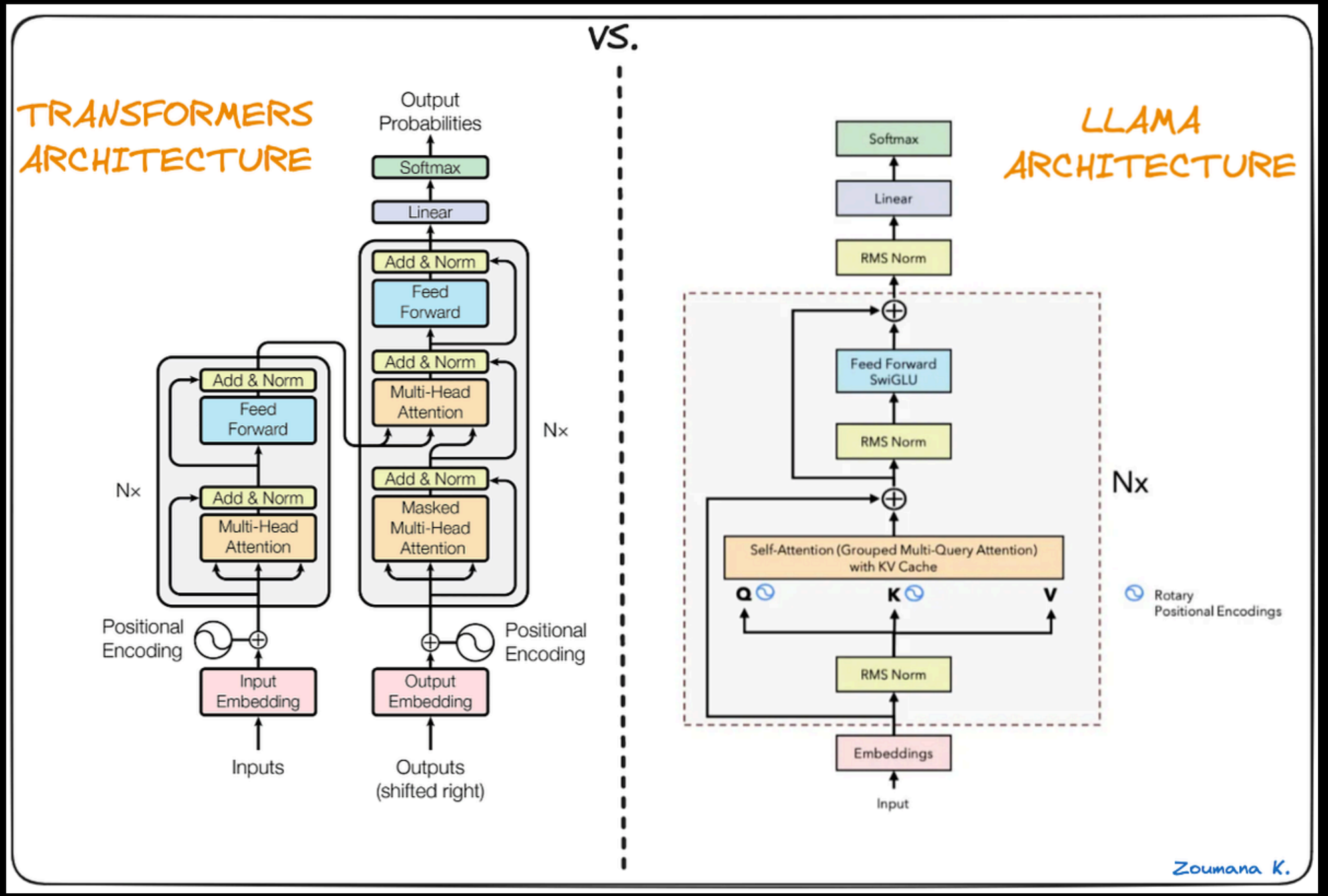
LLAMA models

Before LLAMA (pre 2023) Access to strong models like GPT 3 was limited, closed source and expensive.

Goal of Meta AI is provide open-weight, efficient LLMs for research and innovation

LLAMA 1, LLAMA 2 , LLAMA 3, and so on are all decoder only transformers




VS.
TRANSFORMERS ARCHITECTURE
Output Probabilities
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Nx
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Nx
Positional Encoding
Positional Encoding
Input Embedding
Output Embedding
Inputs
Outputs (shifted right)
LLAMA ARCHITECTURE
Softmax
Linear
RMS Norm
Feed Forward SwiGLU
RMS Norm
Self-Attention (Grouped Multi-Query Attention) with KV Cache
Q
K
V
RMS Norm
Nx
Rotary Positional Encodings
Embeddings
Input
Zoumana K.

SwiGLU

- The weights inside SwiGLU are absolutely updated during training. SwiGLU is not a fixed function—it contains learnable weight matrices, and those weights are trained using backpropagation just like attention weights or embedding weights.
- ReLU and GELU treat every feature identically, the decision to pass or block depends only on the value itself not on another learned projection

$$SwiGLU(x) = (xW1) \otimes Swish(xW2)$$

$$Swish(x) = x \cdot \sigma(x)$$

$$= (xW1) \otimes ((xW2) \cdot \sigma(xW2))$$

Instead of 2 weight matrices, now uses 3.

RMS Norm

- **Root mean square Normalization is a lighter, more stable to layer norm, commonly used in modern LLMs**

$$RMSnorm(x) = \frac{x}{RMS(x)} g$$

g is learned parameter

$$RMS(x) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} {x_i}^2 + \epsilon}$$

RMS Norm

$$\begin{bmatrix} 4 & -2 & 1 \\ 3 & 0 & -1 \end{bmatrix}$$

For first row

$$RMS(row1) = \sqrt{\frac{1}{3}(4^2 + -2^2 + 1^2)} = \sqrt{7}$$

$$RMSnorm(row1) = \frac{row1}{RMS(row1)}$$

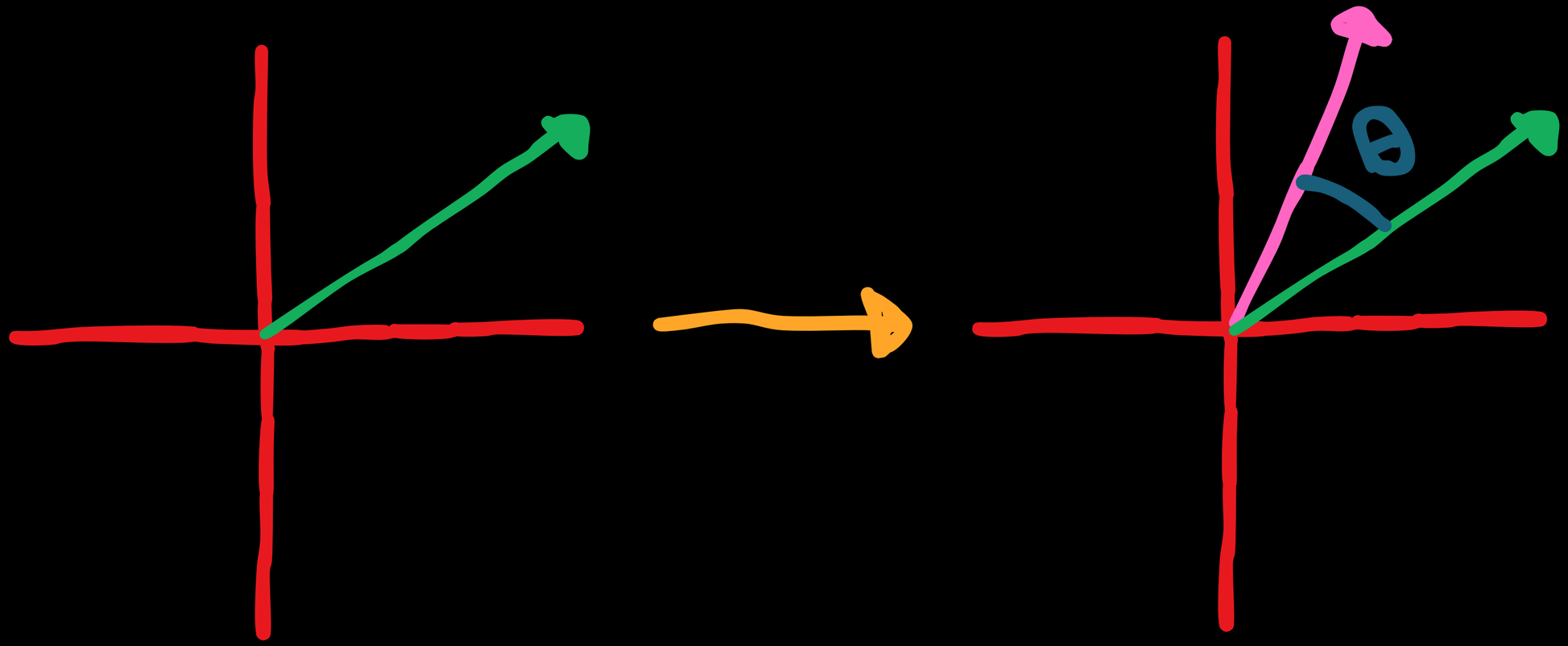
$$= \begin{bmatrix} \frac{4}{\sqrt{7}} & \frac{-2}{\sqrt{7}} & \frac{1}{\sqrt{7}} \end{bmatrix}$$

You can then apply scalar factor

ROPE

Rotary positional encoding

Secret behind LLAMA and GPT success



ROPE

Rotary positional encoding

Secret behind LLAMA and GPT success

$$\begin{bmatrix} x_{new} \\ y_{new} \end{bmatrix} = \begin{bmatrix} \cos(\theta_p) & -\sin(\theta_p) \\ \sin(\theta_p) & \cos(\theta_p) \end{bmatrix} \begin{bmatrix} x_{old} \\ y_{old} \end{bmatrix}$$

ROPE

Rotary positional encoding

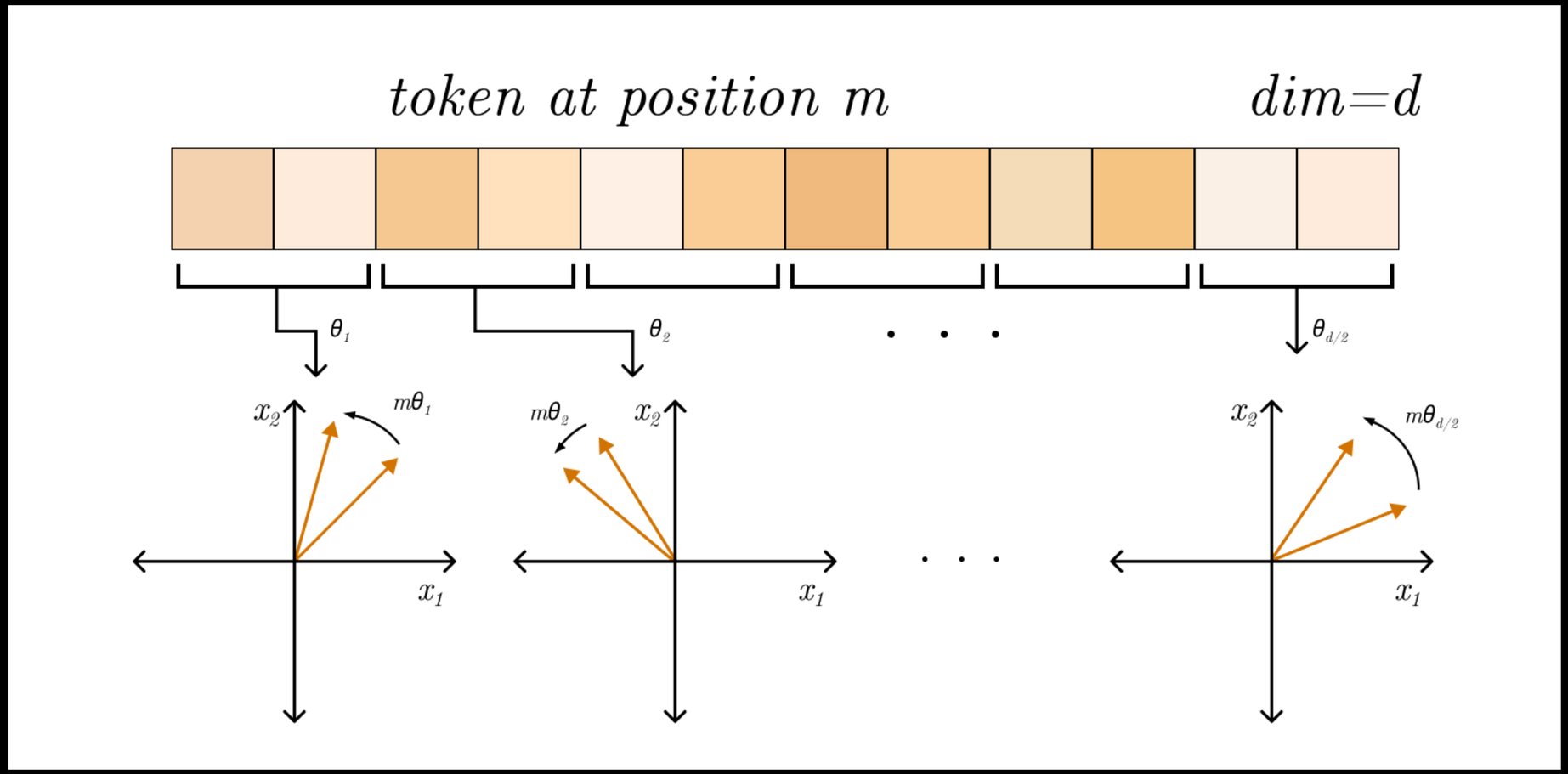
Secret behind LLAMA and GPT success

$$Z_P = Z . e^{j\theta_p}$$

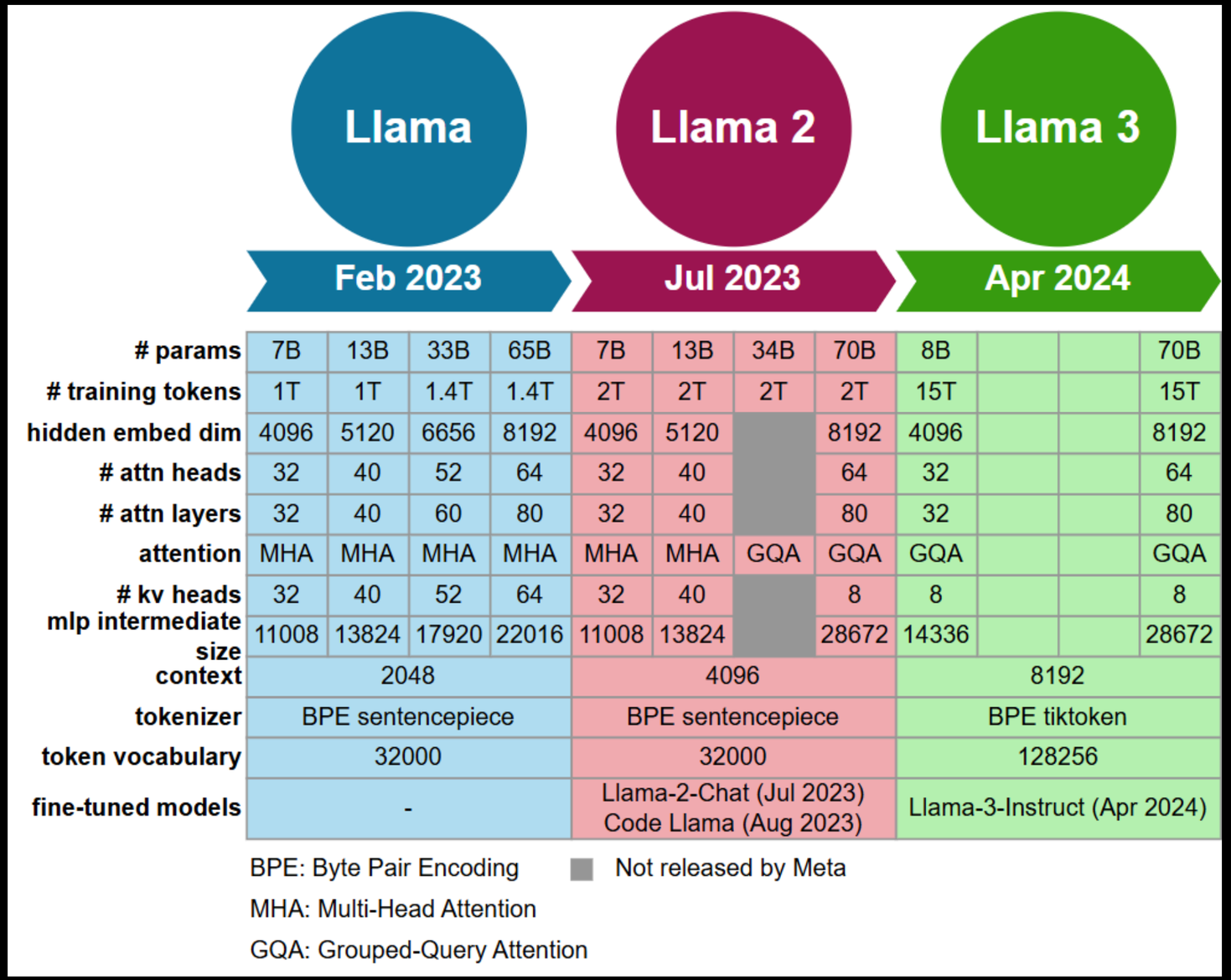
ROPE

Rotary positional encoding

Secret behind LLAMA and GPT success



LLAMA models



# params	7B	13B	33B	65B	7B	13B	34B	70B	8B			70B
# training tokens	1T	1T	1.4T	1.4T	2T	2T	2T	2T	15T			15T
hidden embed dim	4096	5120	6656	8192	4096	5120		8192	4096			8192
# attn heads	32	40	52	64	32	40		64	32			64
# attn layers	32	40	60	80	32	40		80	32			80
attention	MHA	MHA	MHA	MHA	MHA	MHA	GQA	GQA	GQA			GQA
# kv heads	32	40	52	64	32	40		8	8			8
mlp intermediate size	11008	13824	17920	22016	11008	13824		28672	14336			28672
context	2048				4096				8192			
tokenizer	BPE sentencepiece				BPE sentencepiece				BPE tiktoken			
token vocabulary	32000				32000				128256			
fine-tuned models	-				Llama-2-Chat (Jul 2023) Code Llama (Aug 2023)				Llama-3-Instruct (Apr 2024)			

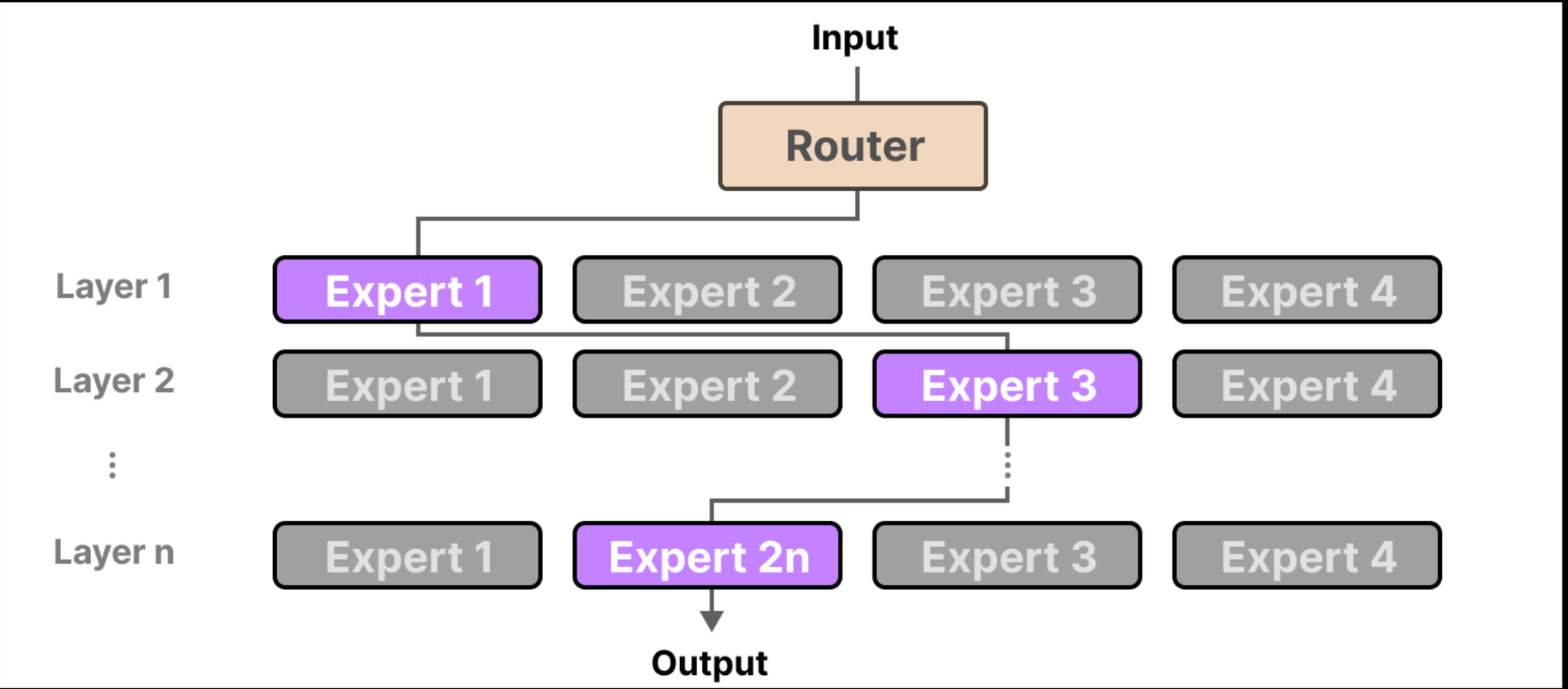
BPE: Byte Pair Encoding ■ Not released by Meta

MHA: Multi-Head Attention

GQA: Grouped-Query Attention

How can you make a model extremely large (for intelligence) without making it extremely slow (for computation)?

Mixture of Experts (MoE)



Mixture of Experts (MoE)

- Instead of one big feed-forward network, we build multiple smaller feed-forward networks. These are called “experts”.
- Now imagine we have N experts, Instead of sending x to all of them, we send x to only a few usually the top k experts (often k = 1 or 2)
- A small routing network (called the gate) decides which experts should handle x.

Mixture of Experts (MoE)

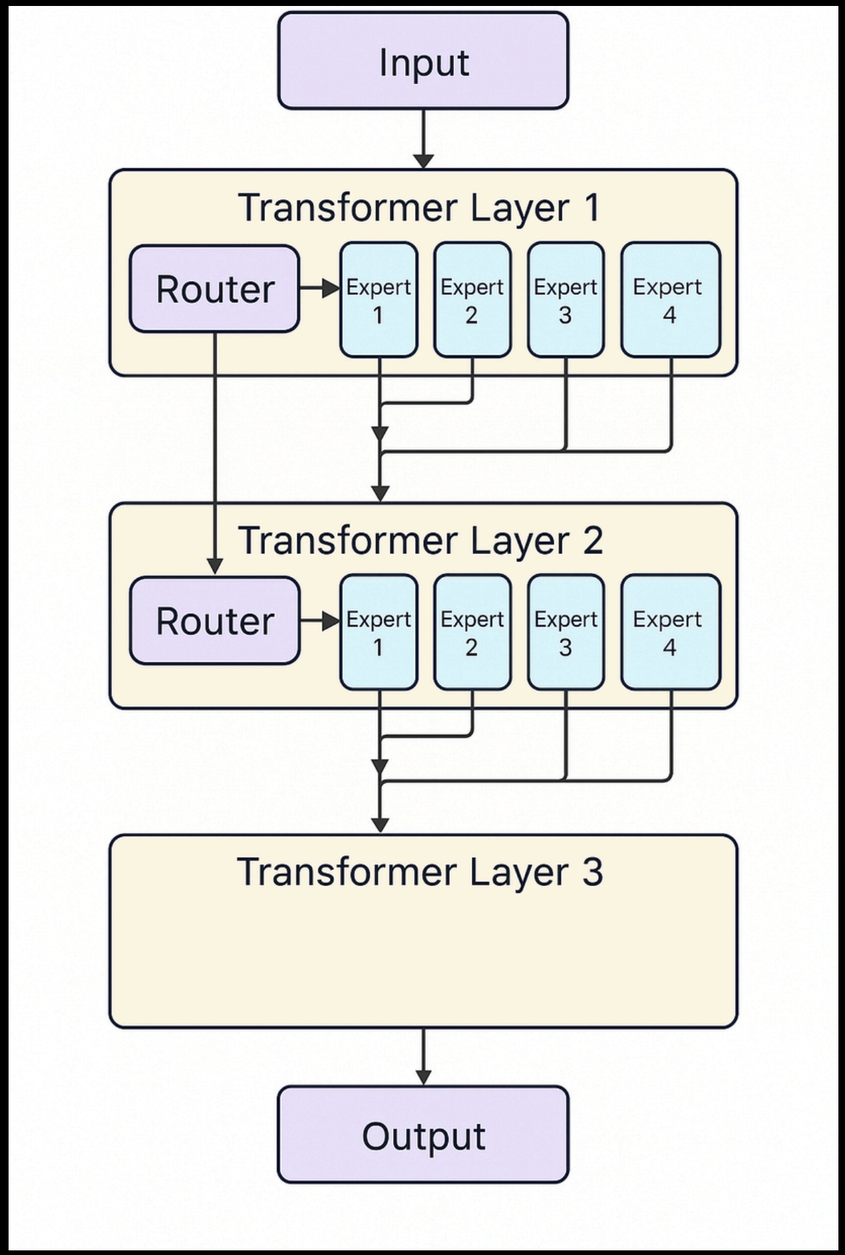
Example :

- **Total experts = 64**
- **Each expert has 100M parameters**
- **Total parameters = 6.4B**

But if k = 2, only 2 experts run per token.

So computation cost ≈ 200M parameters

But total capacity = 6.4B parameters.



Mixture of Experts (MoE)

They are usually just feed-forward layers.
Nothing mystical.
No tiny GPT inside.
No agent.
Just a two-layer MLP.

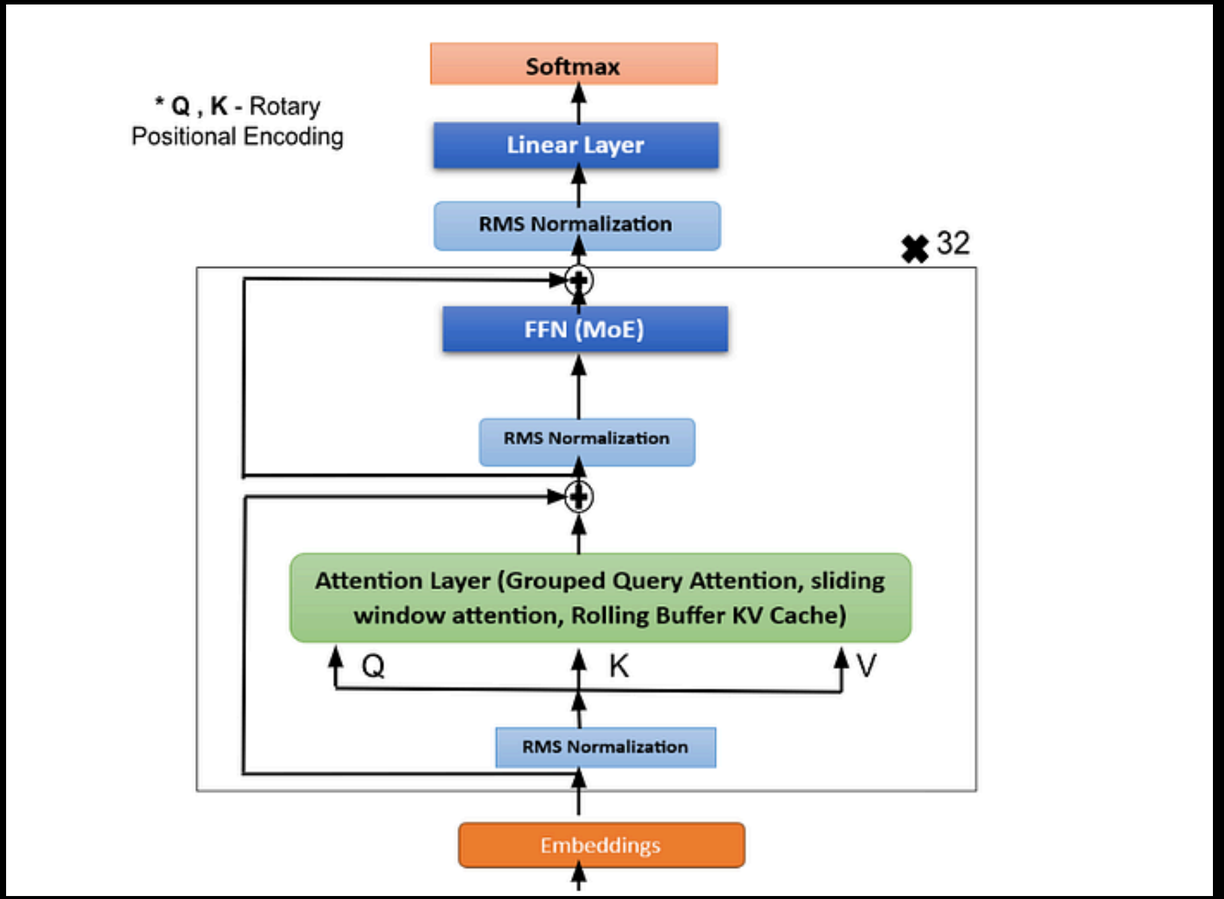
One expert might specialize in:
Code
Math reasoning
Rare words
Certain syntactic patterns
Not because we told it to.
Because gradient descent pushes it there.

Mistral Series

Mistral AI is a research company founded in 2023 in France by former researchers from DeepMind and Meta. Their goal is to build:

- open-weight LLMs
- highly efficient architectures
- models that can run locally or in enterprise environments

Their models are often open-source or open-weight, unlike many closed models.



Mistral Series

Generation 1: Mistral-7B (2023)

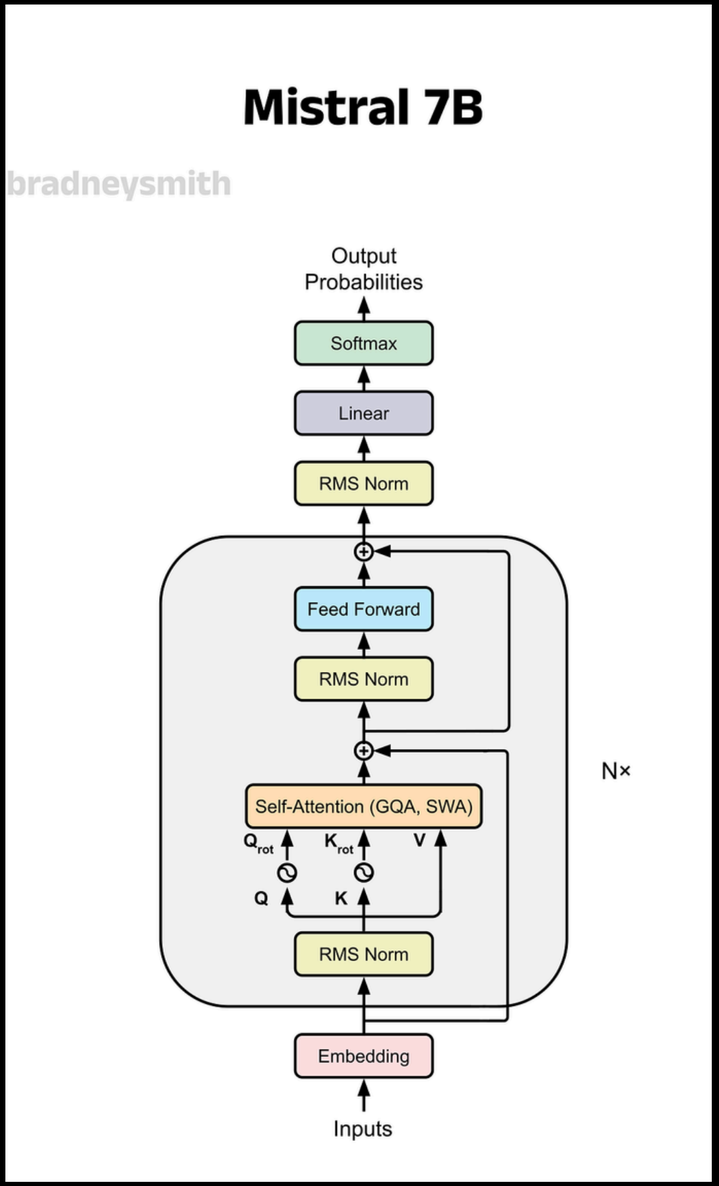
This was their first model.

Features:

- **7 billion parameters**
- **Transformer decoder**
- **uses Grouped Query Attention (GQA)**
- **uses Sliding Window Attention (SWA)**
- **more efficient than larger models**

It outperformed larger models like LLaMA-13B in benchmarks.

This shocked the AI community because it showed: smaller models can outperform larger ones with better architecture.



Mistral Series

Generation 2: Mixtral series (Mixture of Experts)

Examples:

- **Mixtral-8×7B**
- **Mixtral-8×22B**

These use Mixture-of-Experts (MoE).

Example:

Mixtral-8×7B contains:

- **8 experts**
- **total 45B parameters**
- **but only ~13B active per token**

This gives:

- **performance of large model**
- **speed of small model**

Mistral Series

Generation 3: Mistral Large, Medium, Small

These models are categorized by size and power.

Examples:

- **Mistral Small**
- **efficient**
- **enterprise deployment**

Mistral Medium

- **stronger reasoning**
- **multimodal capabilities**

Mistral Large

- **flagship model**
- **~123B parameters**
- **strong reasoning and multilingual ability**

Mistral Series

Generation 4: Specialized models

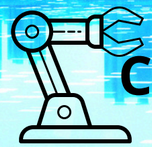
Mistral built specialized models for specific tasks.

Examples:

- **Codestral : optimized for code generation**
- **Mathstral : optimized for math reasoning**
- **Pixtral : multimodal (vision + text)**
- **Voxtral : speech/audio model**
- **Devstral : optimized for software engineering**

These models allow task-specific optimization.

Thank You!



Cellula Technologies